

233 lines (173 loc) · 8.71 KB

Twitter Sentiment Analysis — Apple & Google Products

Author: Mercy Mwangi
Date: June 2026
Topic: NLP Classification + Neural Networks + Model Interpretability
Phase: 4 Project

Project Overview

Business Problem

1. Apple and Google receive thousands of tweets daily about their products, how do brand teams manually read & categorise this data?.
2. What can be done to eliminate delays in responding to negative brand signals that may be caused by feature fails or software updates?.
3. How do we deal with inability to measure brand perception objectively?.
4. What can we do to have distinction between product lines?
5. How do we handle class imbalance hiding the real business problems?

This project builds NLP sentiment classifiers for tweets about Apple and Google products using a systematic three-tier modelling pipeline:

- **Phase A:** Binary classification (Positive vs Negative) — validates the pipeline
- **Phase B:** Multiclass classification (Positive / Neutral / Negative) — traditional ML
- **Phase C:** Neural Networks — MLP Classifier and Bidirectional LSTM

Dataset: **9,093 human-labelled tweets** from CrowdFlower (data.world). After cleaning: **8,892 tweets**.

Repository Structure

├─ index.ipynb	# Main Jupyter Notebook (10 steps)	
├─ Twitter_Sentiment_NLP.pptx	# Presentation (13 slides)	
├─ judge-1377884607_tweet_product_company.csv	# Raw dataset	
└─ README.md	# This file	

Notebook Flow — 10 Steps (Systematic Order)

Step	Section	Description
1	Import Libraries	ML, NLP, TensorFlow/Keras, LIME
2	Load & Explore	Shape, dtypes, stats, nulls, class distribution chart
3	Data Cleaning	Drop duplicates, nulls, ambiguous labels → <code>df_cleaned</code> (8,892 rows)
4	EDA	Histogram, scatter plot, line plot, product × sentiment heatmap
5	Data Preparation	TF-IDF, Keras Tokenizer, LabelEncoder, stratified 80/20 split
6A	Binary ML	NB baseline → LR → Random Forest (best binary model)
6B	Multiclass ML	NB baseline → LR Balanced (best traditional multiclass)
6C	Neural Networks	MLP Classifier → Bidirectional LSTM + learning curves
7	Interpretability	LIME local explanations + LR coefficient chart (global)
8–10	Results	Key Insights → Evaluation → Conclusion & Recommendations

Note: All three modelling phases (6A, 6B, 6C) run sequentially before interpretability (Step 7), ensuring a clean systematic flow.

Dataset

Property	Value
Source	CrowdFlower via data.world
Raw rows	9,093
Columns	<code>tweet_text</code> , <code>product</code> , <code>sentiment</code>
After cleaning	8,892 rows
Class distribution	Neutral 60% · Positive 33% · Negative 6%
Key challenge	Severe class imbalance — only ~6% negative tweets

Data Preparation

Text Cleaning (NLP Pipeline)

- Remove URLs, `@mentions` , `#hashtags`
- Lowercase all text
- Strip punctuation and digits
- Remove NLTK stop-words
- Lemmatize tokens (`WordNetLemmatizer`)
- TF-IDF** encoding — unigrams+bigrams, max 5,000 features (used by NB, LR, RF, MLP)
- Keras Tokenizer** + `pad_sequences` max length 80 (used by Bi-LSTM)

Encoding

- `product` → `LabelEncoder`
- `sentiment` → `LabelEncoder` + `to_categorical` (for LSTM)

- All steps wrapped in `scikit-learn Pipeline` to prevent data leakage

Modelling Results

Phase A — Binary (Positive vs. Negative)

Model	Accuracy	F1 (Positive)
Naïve Bayes (Baseline)	84.6%	0.916
Logistic Regression	85.5%	0.920
Random Forest ✓ Best Binary	88.6%	0.936

Phase B — Multiclass: Traditional ML

Model	Accuracy	Weighted F1	Neg. Recall
Naïve Bayes (Baseline)	66.3%	0.606	11%
LR Balanced ✓ Best Traditional	63.8%	0.649	58%

`class_weight='balanced'` raised Negative recall from 11% → 58%.

Phase C — Neural Networks

Model	Package	Architecture	Accuracy	Weighted F1
MLP ✓ Best Multiclass	scikit-learn	TF-IDF → Dense(128,64) → Softmax(3)	67.5%	0.659
Bidirectional LSTM	TensorFlow/Keras	Embedding → BiLSTM(64) → MaxPool → Dense(64) → Softmax(3)	67.1%	0.638

LSTM trained with `EarlyStopping(patience=3)` — stopped at 5/15 epochs; best weights restored automatically.

All Models at a Glance (Multiclass Weighted F1)

Naïve Bayes	0.606	■■■
LR Balanced	0.649	■■■■■
MLP ★	0.659	■■■■■■■
Bi-LSTM	0.638	■■■■■

Evaluation

Final Model Summary

Task	Phase	Best Model	Accuracy	Weighted F1
Binary Pos/Neg	6A	Random Forest	88.6%	0.936
Multiclass — Traditional	6B	LR Balanced	63.8%	0.649
Multiclass — Neural Net	6C	MLP Classifier	67.5%	0.659

Task	Phase	Best Model	Accuracy	Weighted F1
Multiclass — Deep Learning	6C	Bi-LSTM	67.1%	0.638

Metric Justification

- **Weighted F1** over accuracy — a model predicting "Neutral" every time scores 60% accuracy yet is useless
- Negative recall tracked separately — highest business cost class
- Same stratified 80/20 test set used for all models — fair comparison

Model Interpretability — LIME (Step 7)

Applied **after all three modelling phases** — consistent with the notebook flow.

- **Local (LIME):** explains individual tweet predictions word by word
- **Global (LR coefficients):** reveals which words the model associates with each class across all tweets
- Top positive signals: *love, great, awesome, new ipad*
- Top negative signals: *fail, problem, crash, fix, lose*

Key Insights

1. 6% negative tweets — severe imbalance; weighted F1 is the correct metric
2. **Negative tweets are longer (~109 chars vs ~106)** — users elaborate when frustrated
3. iPad and Apple generate most positive sentiment; Google leans neutral
4. Bigrams improved binary accuracy from 84.6% to 88.6%
5. **MLP achieves best multiclass weighted F1 (0.659)** — learns non-linear TF-IDF combinations
6. Bi-LSTM competitive (67.1%) but needs more data to outperform traditional ML
7. Early stopping prevented overfitting — LSTM stopped at 5/15 epochs
8. LIME confirms: *love/great/awesome* → Positive; *fail/crash/problem* → Negative
9. `class_weight='balanced'` raises Negative recall 11% → 58%
10. Sarcasm and ambiguity remain unsolved — BERT is the recommended next step

Recommendations

Priority	Action	Rationale
High	Deploy binary Random Forest	88.6% accuracy — production-ready now
High	Deploy MLP for multiclass dashboard	Best weighted F1 (0.659)
High	Collect more negative tweets	Only 6%; recall will improve
Medium	Retrain quarterly	Vocabulary evolves with product cycles
Medium	Build Twitter API pipeline	Automate brand monitoring
Low	Fine-tune BERT / RoBERTa	Best path to sarcasm detection

Libraries & Tools

Library	Purpose
pandas , numpy	Data manipulation
matplotlib , seaborn	Visualisation
nltk	Stop-words, lemmatization
scikit-learn	TF-IDF, NB, LR, RF, MLP, pipelines, evaluation
tensorflow / keras	Bidirectional LSTM deep learning
lime	Local model interpretability

How to Run

```
git clone https://github.com/Mercy1989/Twitter_Sentiment.git
cd Twitter_Sentiment
pip install pandas numpy matplotlib seaborn nltk scikit-learn tensorflow lime
jupyter notebook index.ipynb
```

Use Kernel → Restart & Run All to execute all cells in order.

Presentation

Twitter_Sentiment_NLP.pptx — 13 slides matching the notebook flow:

Slide	Content
1	Title — Mercy Mwangi, June 2026
2	Project Structure & Notebook Flow (visual diagram)
3	Steps 2 & 3 — Data Understanding & Cleaning
4	Step 5 — Data Preparation
5	Steps 6A & 6B — Traditional ML Models
6	Step 6C — Neural Network Models (MLP + Bi-LSTM)
7	Step 6C — LSTM Training History (learning curves)
8	All Models Comparison
9	Step 7 — LIME Interpretability
10	Step 8 — Key Insights
11	Step 9 — Evaluation
12	Conclusion, Recommendations & Next Steps
13	Thank You / Closing

Preview Code Blame



Raw

